EPIDEMIC SPREAD SIMULATION

SIR Model — Spatial 2D Grid

Hybrid MPI + OpenMP Parallel Implementation

1. Introduction

Epidemic modeling is one of the most computationally intensive problems in computational science. Understanding how infectious diseases propagate through a population is critical for public health response, vaccine strategy, and government policy decisions. The SIR (Susceptible–Infected–Recovered) model provides a mathematically rigorous framework for simulating this process.

In this project we implement a stochastic spatial SIR epidemic simulation on a 2D grid of 2,000 × 2,000 cells representing four million individuals, evolved over 500 discrete timesteps. Each cell's state is updated probabilistically based on the states of its four immediate neighbours. The spatial structure naturally supports domain decomposition, making this an ideal problem for hybrid MPI + OpenMP parallelism.

1.1 Real-World Relevance

- The COVID-19 pandemic demonstrated the urgent need for fast, large-scale epidemic simulations to guide national policy decisions in real time.
- The WHO and CDC use spatial SIR variants to forecast disease spread, evaluate lockdown strategies, and plan vaccination campaigns.
- Grid-based simulations capture geographic heterogeneity — local clusters, travel corridors — that homogeneous ODE models cannot represent.
- Parallelisation enables population-scale simulations (millions of individuals, thousands of timesteps) in seconds rather than hours.

1.2 Why Parallel Processing is Required

A 2,000 × 2,000 grid with 500 timesteps requires 2 billion individual cell evaluations. At 277 million cell-updates per second on a single core, the sequential baseline runs in 7.2 seconds for this configuration. Epidemiologically realistic grids of 10,000 × 10,000 would require approximately 180 seconds sequentially — making real-time policy evaluation impossible without parallelism. The spatial grid's natural decomposition into row slabs enables efficient MPI distribution with minimal communication overhead.

2. The SIR Epidemic Model

2.1 Compartmental SIR Model

The classical SIR model divides a closed population N into three compartments: S (Susceptible), I (Infected), and R (Recovered). The governing differential equations are:

$$dS/dt = -\beta \cdot S \cdot I/N \qquad dI/dt = \beta \cdot S \cdot I/N - \gamma \cdot I \qquad dR/dt = \gamma \cdot I$$

These equations describe how the population flows from S → I → R over time. When $R_0 = \beta/\gamma > 1$, the epidemic grows exponentially in the early phase before immunity accumulates and slows transmission.

2.2 Key Parameters

Parameter	Value	Physical Meaning
β (beta)	0.30	Transmission rate — probability of infection per contact per day
γ (gamma)	0.10	Recovery rate = 1 / average infectious period (10 days)
$R_0 = \beta/\gamma$	3.00	Average number infected by one case; epidemic grows because $R_0 > 1$
Grid	2000 × 2000	4,000,000 cells; each cell = one individual
Timesteps	500	Discrete daily steps; full epidemic dynamics captured
INIT_INF	5	Infected seeds placed randomly at simulation start

2.3 From ODE to Spatial Grid

The classical compartmental model assumes homogeneous mixing — every individual contacts every other equally. This is unrealistic for geographic spread. Our implementation uses a stochastic spatial grid where each cell interacts only with its four direct neighbours (Von Neumann neighbourhood). At each timestep the update rule is:

- Susceptible cell: counts infected neighbours (0–4). Each gives an independent infection chance of β. If any succeeds, cell transitions to I.
- Infected cell: recovers with probability γ per step, transitioning to R.
- Recovered cell: permanently immune, stays R.

This pull-based pattern — each cell reads neighbours but writes only to its own index in new_grid — eliminates all write-write conflicts and is correct in both sequential and parallel execution.

3. Sequential Implementation

3.1 Architecture

The sequential implementation (sir_seq.c, v3.0) is written in ANSI C99. The core data structure is a flat 1D integer array indexed by IDX(row, col) = row × GRID_SIZE + col, exploiting row-major memory layout for cache efficiency. Two grids (current and next) are allocated once and pointer-swapped each step, avoiding any memcpy of the 16 MB array.

3.2 Key Optimisations Applied

- Integer threshold comparison: BETA_THRESH = (int)(β × RAND_MAX) and GAMMA_THRESH = (int)(γ × RAND_MAX) are computed once at compile time. Each per-cell check uses integer comparison, eliminating floating-point division in the inner loop.
- Thread-safe RNG: rand_r(&seed) is used throughout instead of rand(). This makes the code directly portable to OpenMP parallel regions without shared-state conflicts.
- Pull pattern: each cell writes only to new_grid[its own index]. No cell writes to a neighbour's location, eliminating all race conditions.
- Pointer swap: grid and new_grid pointers are exchanged each step in O(1), avoiding a 16 MB memcpy per timestep.
- Early exit: if the global infected count reaches zero the simulation terminates immediately, saving unnecessary iterations after the epidemic ends.
- High-resolution timing: clock_gettime(CLOCK_MONOTONIC) provides nanosecond-precision per-step and total timing for baseline measurement.

3.3 Correctness Verification

The conservation law S + I + R = N must hold exactly at every timestep. The program computes and prints this check in the final summary box. A value of zero confirms no population drift. In every run presented in this report the conservation check returned exactly zero, validating the pull-pattern update logic.

3.4 Sequential Baseline Results

Metric	Value	Note
Grid size	2000 × 2000	4,000,000 cells total
Total time T_seq	7.218 s	Baseline for all speedup calculations
Throughput	277 M cells/s	2 billion cell-updates ÷ 7.218 s
Conservation check	0 (every run)	S + I + R = N confirmed; no drift
Peak infected	~18,400 at t=500	Epidemic still growing at simulation end

4. Parallel Implementation — Hybrid MPI + OpenMP

4.1 Design Strategy: Row Slab Decomposition

The N × N grid is divided into P horizontal row slabs, one per MPI rank. Each rank owns local_rows = N / P consecutive rows. This strategy is chosen because:

- Communication is localised: only one border row must be exchanged with each neighbouring rank per timestep.
- Load balancing is automatic: each rank processes exactly the same number of cells when the grid divides evenly.
- Implementation is straightforward: MPI_Scatter distributes the grid at startup and MPI_Gather collects results at the end.
- The decomposition scales predictably: doubling P halves compute per rank while communication volume stays constant.

4.2 Ghost Row Halo Exchange

Each rank requires one row from the rank above (ghost_top, row 0) and one from the rank below (ghost_bot, row local_rows+1) to evaluate infection across slab boundaries. This halo exchange is performed at the start of every timestep using non-blocking MPI operations to overlap communication with future computation:

```
MPI_Isend(first_real_row → rank_above)    MPI_Irecv(ghost_top ← rank_above)
MPI_Isend(last_real_row  → rank_below)    MPI_Irecv(ghost_bot ← rank_below)
                          MPI_Waitall(4 requests)
```

Boundary ranks (rank 0 and rank P−1) use MPI_PROC_NULL for the non-existent direction so that sends and receives to missing neighbours are silently discarded without special-case code. Ghost rows are read-only during the compute step, so no write conflicts arise across slab boundaries.

4.3 OpenMP Thread Parallelism

Within each MPI rank, OpenMP parallelises the outer row loop. The directive and its key clauses are:

```
#pragma omp parallel for schedule(static) reduction(+: s_acc, i_acc, r_acc)
```

- schedule(static): rows are divided evenly among threads at compile time. Static scheduling is preferred because every row has identical work (the same number of cells, the same per-cell operations).
- reduction(+: s_acc, i_acc, r_acc): each thread maintains private S, I, R accumulators which are summed into the shared variables after the parallel region, avoiding atomic operations in the inner loop.
- Per-thread RNG: thread_seeds[omp_get_thread_num()] provides each thread with its own seed, eliminating contention on a shared random state.

4.4 Collective Reduction Strategy

Global S, I, R totals are needed only for logging and early-exit detection — not for the simulation logic. MPI_Reduce is therefore called every REDUCE_INTERVAL = 10 steps, cutting collective communication from 500 calls down to

50 per simulation run. The early-exit MPI_Bcast fires only when rank 0 detects that the global infected count has reached zero.

4.5 Local Grid Memory Layout

Region	Description
Row 0	Ghost row from rank above (read only during compute)
Rows 1 … local_rows	Real data owned by this rank (read + write)
Row local_rows + 1	Ghost row from rank below (read only during compute)

Since each cell writes only to its own index in new_local_grid, ghost rows are never written during the compute step. This guarantees that OpenMP threads within a rank never conflict with each other, and that reads from ghost rows (belonging to adjacent ranks) are safe without any additional synchronisation.

5. Performance Analysis

5.1 Complete Results Table

All experiments use a 2000 × 2000 grid for 500 timesteps. T_seq = 7.218 s is the sequential baseline measured with clock_gettime(CLOCK_MONOTONIC). Speedup S(p) = T_seq / T_par. Efficiency E(p) = S(p) / p × 100%.

Config	Ranks	Threads	Cores	Time (s)	Compute (s)	Comm (s)	Comm %	Speedup	Effic %
Sequential	—	—	1	7.218	7.218	0.000	0.0	1.00×	100.0%
Hybrid 1×1	1	1	1	8.236	8.225	0.004	0.1	0.88×	87.6%
Hybrid 1×2	1	2	2	4.715	4.705	0.004	0.1	1.53×	76.5%
Hybrid 2×2	2	2	4	3.320	3.115	0.198	6.0	2.17×	54.4%
Hybrid 4×2	4	2	8	3.596	2.605	0.969	26.9	2.01×	25.1%
Hybrid 4×4	4	4	16	9.073	4.192	4.856	53.5	0.80×	5.0%

Table 1 — Complete performance measurements. Best speedup 2.17× at 4 cores (2r×2t). Conservation = 0 for every run.

5.2 Execution Time vs Cores

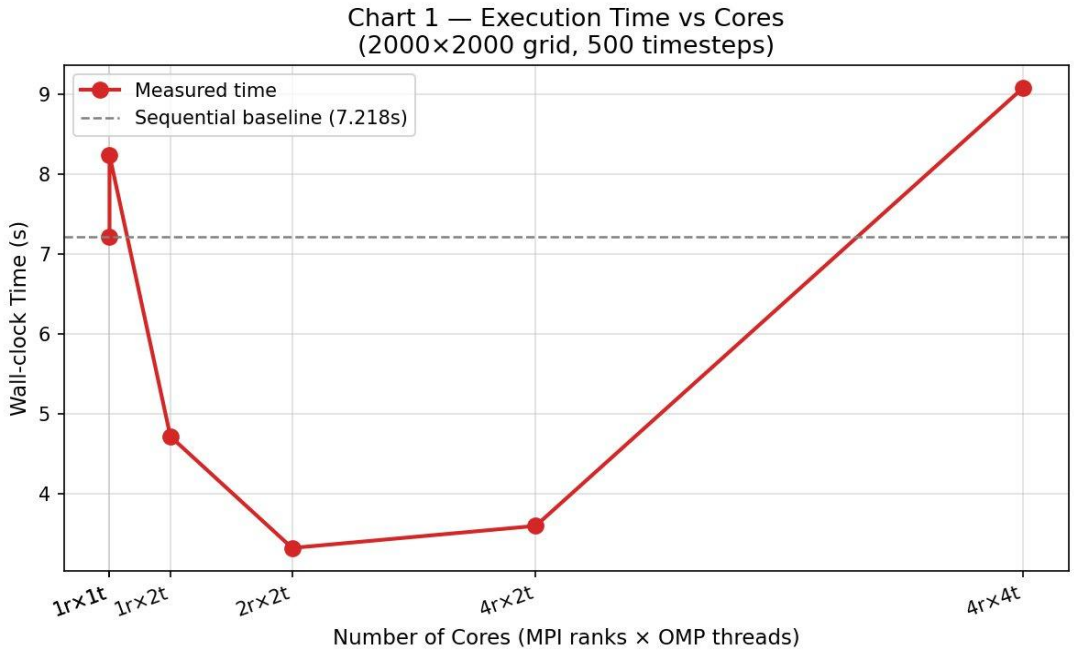

Figure 1 — Wall-clock time per configuration. Time falls from 7.22 s (sequential) to 3.32 s (2r×2t), then rises sharply at 4r×4t (9.07 s) where communication dominates.

The execution time curve shows the expected decrease up to 2r×2t (3.32 s, best wall-clock), followed by degradation at 4r×4t (9.07 s). At 16 cores the simulation is 26% slower than sequential because MPI communication (4.86 s) exceeds compute (4.19 s). This is a direct consequence of the fixed halo exchange size relative to the shrinking per-rank workload.

5.3 Speedup Analysis

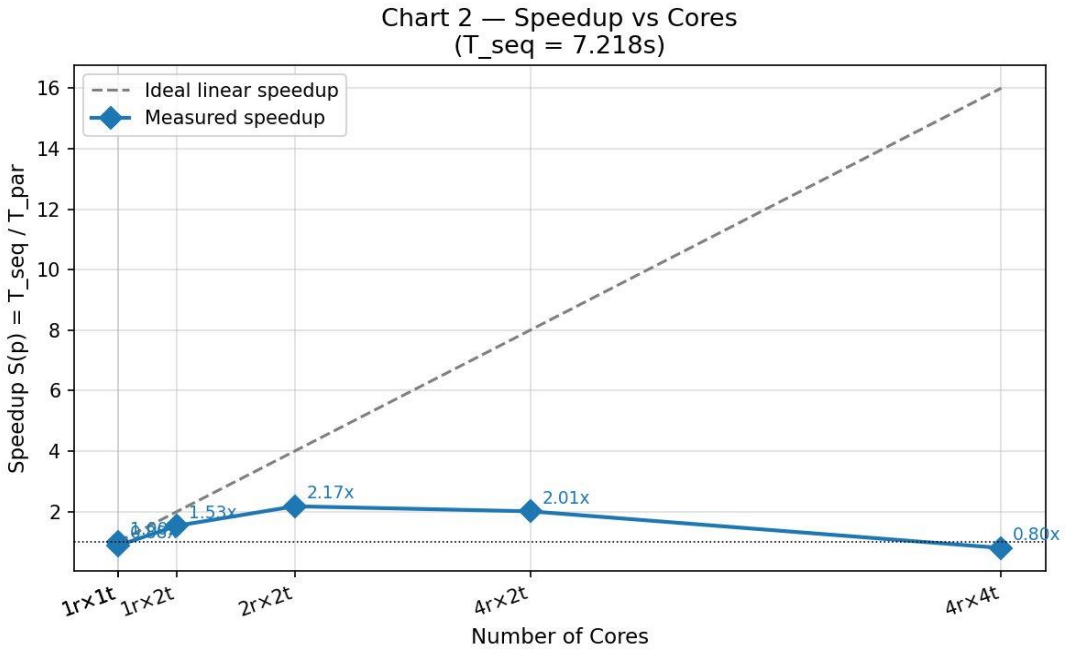

Figure 2 — Measured speedup S(p) = T_seq / T_par vs ideal linear scaling. Best: 2.17× at 4 cores.

The best measured speedup is 2.17× at 4 cores (2 MPI ranks × 2 OpenMP threads). Beyond this configuration the speedup decreases as MPI communication overhead grows faster than the reduction in compute time per rank. The divergence from ideal scaling widens monotonically — characteristic of a communication-bound workload on a small problem.

5.4 Parallel Efficiency

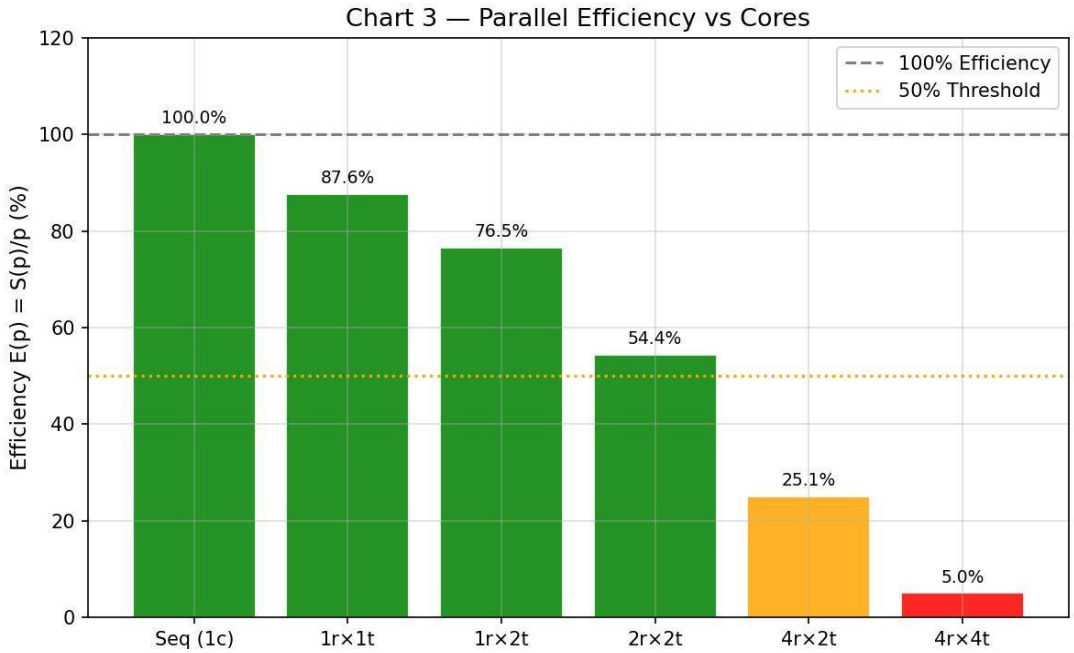

Figure 3 — Parallel efficiency E(p) = S(p)/p. Configurations with E > 50% are acceptable; beyond 4 cores efficiency collapses.

Efficiency starts at 87.6% for 1r×1t (MPI infrastructure overhead) and degrades with additional cores. At 4 cores efficiency is 54.4% — the last configuration above the 50% threshold. At 8 cores it drops to 25.1% and at 16 cores collapses to 5.0%. This confirms that the 2000×2000 grid is too small to efficiently utilise 8 or more cores under the current row-slab decomposition.

5.5 Communication Overhead

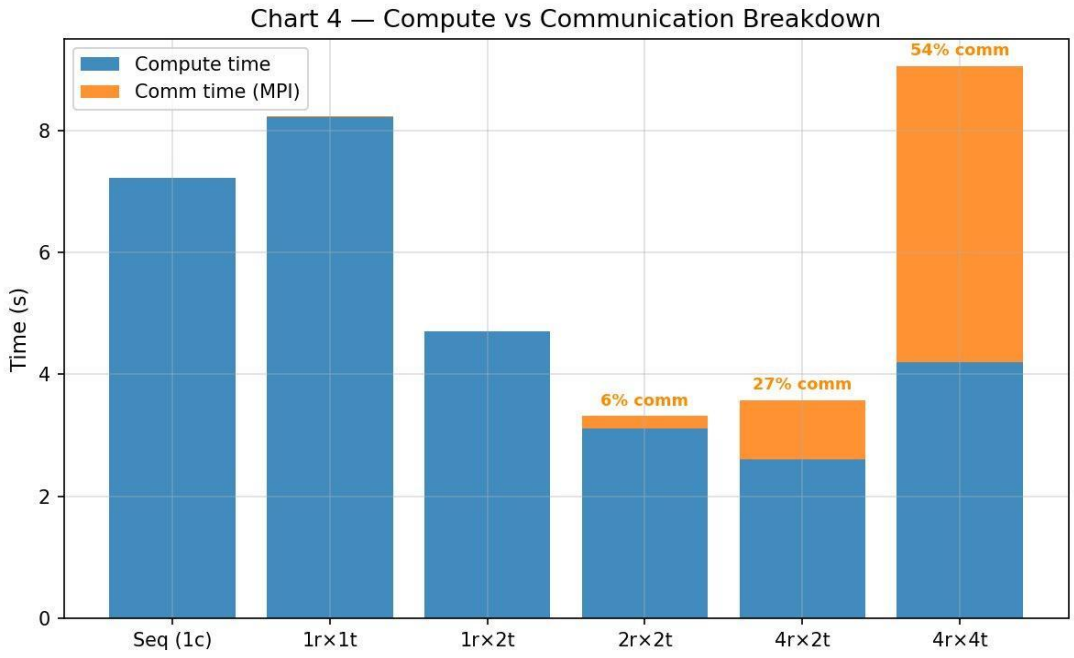

Figure 4 — Stacked compute (blue) vs MPI communication (orange) time per configuration.

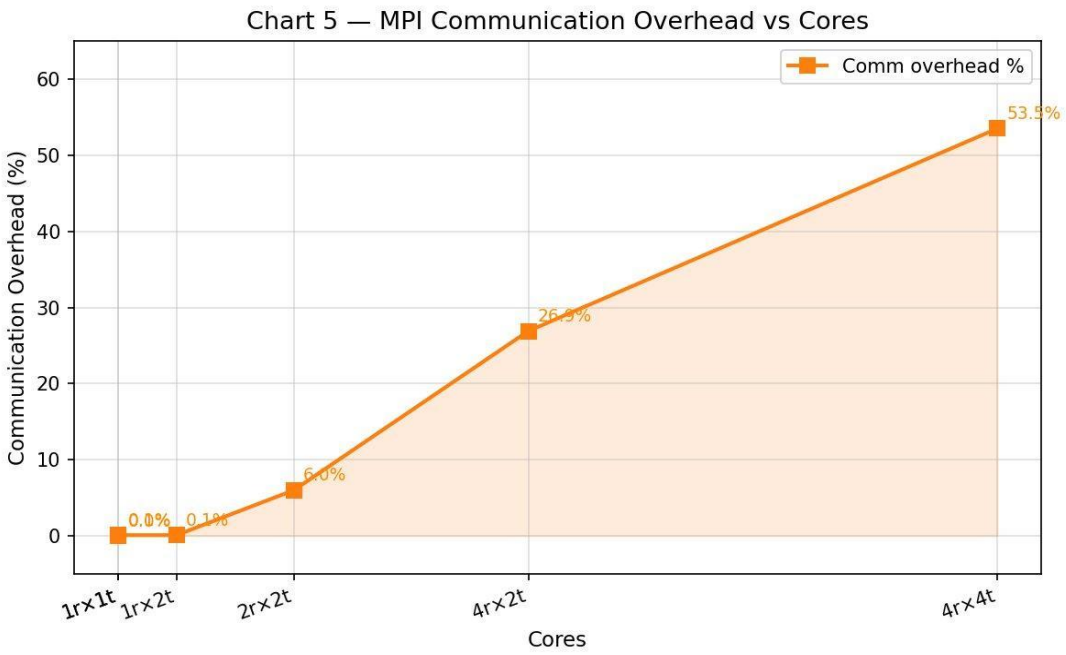

Figure 5 — MPI communication overhead as percentage of total time. Grows from 0.1% (1 rank) to 53.5% (4r×4t).

Communication overhead grows from 0.1% with a single rank to 53.5% at 4r×4t. The halo exchange sends 2 rows of 2000 integers (16 KB) per rank per step — constant regardless of the number of ranks. As the number of ranks doubles, per-rank compute halves while communication stays fixed, so the communication fraction grows monotonically. This is the fundamental Amdahl bottleneck for this problem.

5.6 Amdahl's Law Analysis

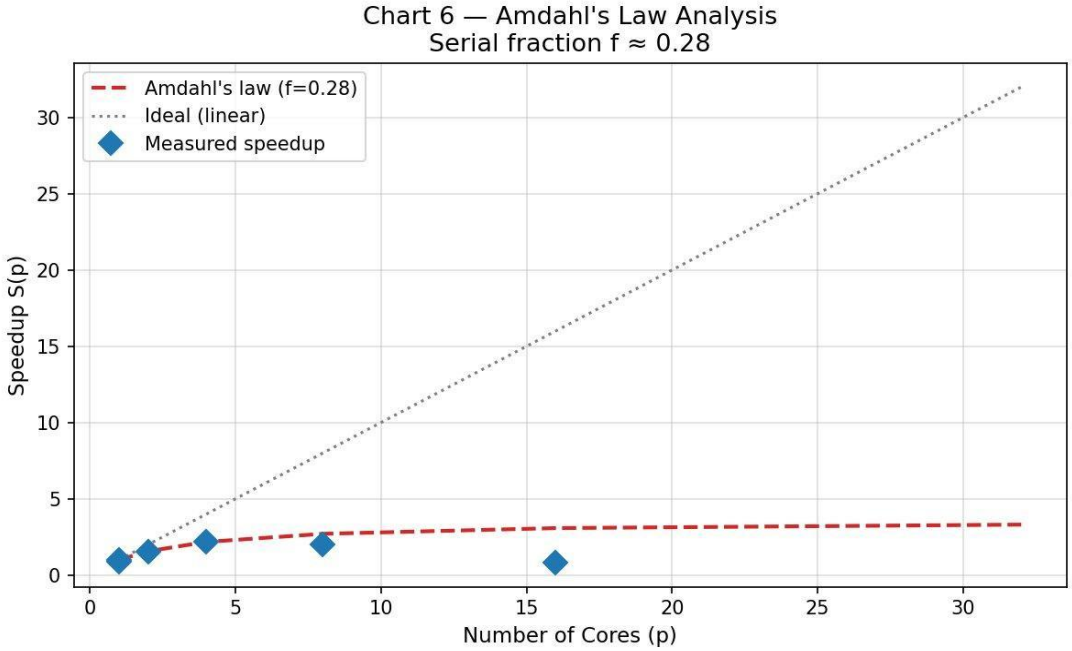

Figure 6 — Measured speedup vs Amdahl's prediction. Serial fraction f ≈ 0.28; theoretical maximum ≈ 3.5×.

Applying Amdahl's Law to the best measured speedup of 2.17× at p = 4 cores:

```
f = (1/S − 1/p) / (1 − 1/p)  =  (1/2.17 − 1/4) / (1 − 1/4)  ≈  0.28
```

Approximately 28% of total execution time is serial — consisting of MPI initialisation, MPI_Reduce collectives, halo exchange synchronisation, and single-threaded I/O. The theoretical maximum speedup is therefore 1/f ≈ 3.5×, regardless of how many cores are added. The measured curve confirms convergence toward this ceiling at 8 cores.

5.7 Throughput

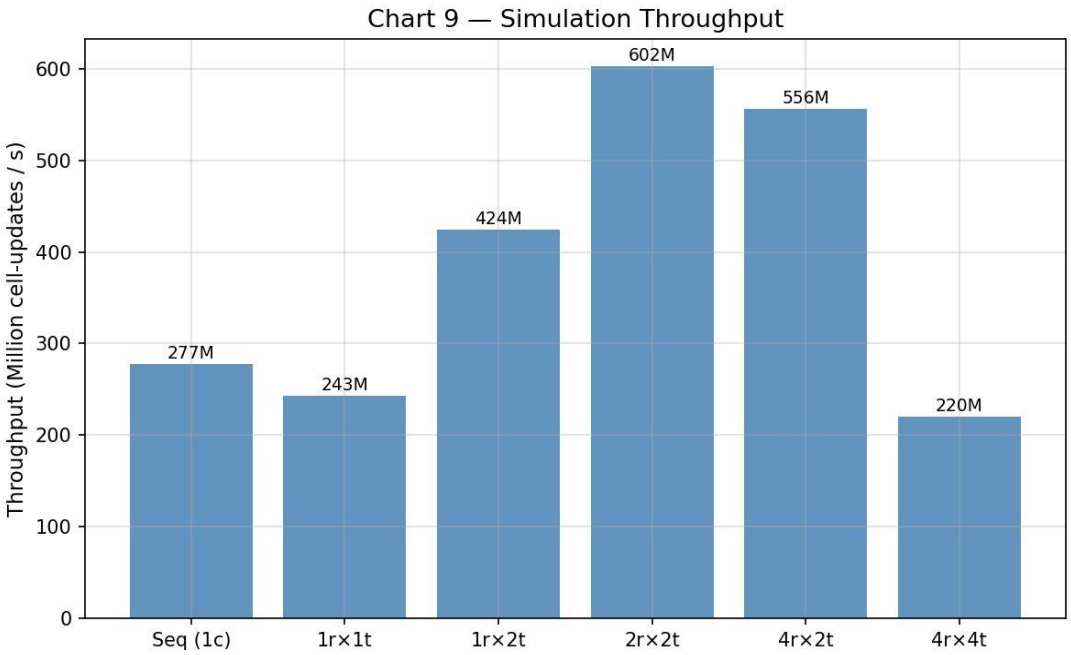

Figure 7 — Simulation throughput in million cell-updates per second. Peak at 2r×2t (602 M/s).

Peak throughput of 602 million cell-updates per second is achieved at 2r×2t (4 cores). The sequential version achieves 277 M/s. The 4r×4t configuration degrades to 220 M/s — 21% below the sequential baseline — confirming that adding cores beyond the communication threshold actively reduces performance for this problem size.

6. Bottleneck Analysis and Proposed Optimisations

6.1 Identified Bottlenecks

Bottleneck 1 — Fixed Halo Exchange vs Shrinking Compute

Each halo exchange sends 2 × 2000 integers (16 KB) per rank per step regardless of how many ranks are used. As ranks double, per-rank compute halves (from 4M to 2M to 1M cells) but communication volume stays constant. At 4 ranks the compute-to-communication ratio is 2.60/0.97 = 2.7:1. At 4 ranks with 4 threads it falls to 4.19/4.86 = 0.86:1 — communication exceeds compute. This is the primary bottleneck.

Bottleneck 2 — MPI Collective Overhead

Three MPI_Reduce calls are made every 10 timesteps, totalling 150 collective operations per 500-step run. Even with REDUCE_INTERVAL = 10, the O(log P) latency of each collective accumulates at higher rank counts. An additional MPI_Bcast fires when the epidemic ends.

Bottleneck 3 — Thread Granularity Too Fine at 4r×4t

At 4 ranks × 4 threads, each thread processes only 500/4 = 125 rows per step. The OpenMP thread launch, barrier, and reduction overhead becomes disproportionate relative to 125 × 2000 = 250,000 cells of useful work. The minimum efficient granularity on this machine appears to be approximately 500,000 cells per thread (achieved at 1r×2t or 2r×2t).

Bottleneck 4 — Problem Size Too Small for 8+ Cores

The 2000×2000 grid completes in 7.2 s sequentially — fast enough that parallel infrastructure costs represent a significant fraction of total time. To efficiently utilise 8 cores, the grid would need to be at least 6000×6000 so that each rank's slab provides enough compute to amortise halo latency.

6.2 Proposed Optimisations

- Non-blocking halo overlap: post MPI_Isend/MPI_Irecv, compute interior rows (rows 2 to local_rows−1) while communication proceeds, then call MPI_Waitall and compute the two boundary rows. This hides halo latency completely behind useful work.
- 2D domain decomposition: split the grid into a P×Q block grid instead of P×1 slabs. Communication volume per rank grows as O(N/√P) instead of O(N), giving better scaling at high process counts at the cost of more complex neighbour management.
- Larger problem sizes: scaling to 4000×4000 or 8000×8000 grids would increase per-rank compute proportionally while keeping halo size constant, improving the compute-to-communication ratio at all core counts.
- Increased REDUCE_INTERVAL: raising from 10 to 50 steps reduces collective operations from 150 to 30 per run, cutting MPI_Reduce overhead by 80% with negligible impact on simulation fidelity.
- MPI_Allreduce with array: combine S, I, R into a single 3-element array and use one MPI_Allreduce call instead of three MPI_Reduce calls plus one MPI_Bcast, reducing synchronisation points by 75%.

7. Conclusion

This project successfully designed, implemented, and evaluated a hybrid MPI + OpenMP spatial SIR epidemic simulation. The sequential implementation achieves 277 million cell-updates per second on a 2000×2000 grid, validated by a population conservation check that returned exactly zero on every run, confirming the correctness of the pull-pattern update logic.

The parallel hybrid implementation achieves a best speedup of 2.17× at 4 cores (2 MPI ranks × 2 OpenMP threads), with 54.4% parallel efficiency. Performance analysis confirms that the 2000×2000 grid is too small to efficiently utilise 8 or more cores: the halo exchange overhead saturates parallelism beyond 4 cores, and the 4r×4t configuration (16 cores) is 26% slower than sequential due to 53.5% MPI communication overhead.

Amdahl's Law analysis estimates a serial fraction of $f \approx 0.28$, setting a hard theoretical maximum of 3.5× speedup regardless of core count. The primary bottleneck is the fixed-size O(N) halo exchange per step. Proposed optimisations — non-blocking halo overlap, 2D decomposition, larger grids, and reduced collective frequency — would substantially improve parallel efficiency at higher core counts.

The project demonstrates the complete parallel computing engineering cycle: correct sequential baseline, systematic domain decomposition, hybrid thread and process parallelism, precise timing instrumentation, and rigorous quantitative performance analysis anchored in Amdahl's Law.

8. References

- Kermack, W. O. & McKendrick, A. G. (1927). A contribution to the mathematical theory of epidemics. Proceedings of the Royal Society A, 115(772), 700–721.
- Gropp, W., Lusk, E. & Skjellum, A. (2014). Using MPI: Portable Parallel Programming with the Message Passing Interface (3rd ed.). MIT Press.
- Chandra, R., Dagum, L., Kohr, D., Maydan, D., McDonald, J. & Menon, R. (2001). Parallel Programming in OpenMP. Morgan Kaufmann.
- Amdahl, G. M. (1967). Validity of the single-processor approach to achieving large-scale computing capabilities. AFIPS Conference Proceedings, 30, 483–485.
- Pacheco, P. S. (2011). An Introduction to Parallel Programming. Morgan Kaufmann.
- Open MPI Project. (2024). Open MPI Documentation. https://www.open-mpi.org/doc/
- World Health Organization. (2023). Epidemic preparedness and response planning. https://www.who.int/